Arquiteturas de Alto Desempenho 2026/2027

First practical assignment — modification of the SHA1 hash of some files

Tomás Oliveira e Silva

1 Introduction

The main purpose of this assignment is to write efficient computer programs that can be used to modify a file so that its SHA1 (Secure Hash Algorithm 1)[1] secure hash will end with at least 32 zero bits. This has to be done for

- a Portable Document FILE (PDF) file, by adding a comment near its end,
- a Portable Network Graphics (PNG) file, by adding a tEXt chunk near its end,
- a Portable Pixmap Format (PPM), by changing the least significant bit of the RGB color components of the pixels near its end.

This should also be done for one extra file type of your choice.

2 The structure of PDF files

PDF files are (mostly) text files. PDF files contain integers, strings, arrays, dictionaries and objects. They can also contain JPEG images (with binary data). The reduce its file size, it is common to compress most of its text sections; you can decompress then using functions of the `zlib` library. PDF files also contain composite objects (each page is one such object). At the end of the PDF file, in a so-called `xref` section, the file offset location of each object is recorded. The last section of a PDF file, the trailer, contains the `startxref` section, which in turn contains the file offset of the `xref` section.

Comments in PDF files begin with a '%' and continue until the end of the line. To modify a PDF file so that its SHA1 hash ends with 32 zero bits one possibility is to add to it a comment in an innocuous place. For example, in the `examples/deti_4_2_37_template.pdf` file this was done just before the start of the `startxref` section:

```
%NNNNNNNNNNNN
startxref
```

A total of 14 extra bytes were added to the file. The twelve 'N' characters will then have to be replaced by an appropriate string in order to try to make the file's SHA1 hash have the desired characteristics. (The teacher used a 12 digit counter for this.)

There exist many online resource you can consult in order to get more information about PDF files. There is, of course, the official PFD specification document, but that may be a little hard to read. There is also, for example, this online PyPDF2 documentation.

3 The structure of PNG files

PNG files are used to store compressed images. The PNG file structure is composed by chunks. The integrity of each chunk can be checked using a 32-bit Cyclic Redundancy Checksum (CRC). Some of the chunks are mandatory, and must appear in specific parts of the file; others are optional. We will be particularly interested in creating a so-called `xEXt` chunk (place it as close as possible near the end of the file), holding a text pattern that we can modify at will. Changing it (and adjusting the chunk's CRC) will allow us to make the file's SHA1 hash have the desired characteristics.

[1] See this wikipedia page or this request for comments document.

4 The structure of PPM files

PNG files are used to store uncompressed images. As they do not use compression, they are particularly easy to work with. We will use the P6 format (RGB data in binary, one byte for each). After the file header, a width × height image is stored pixel by pixel (R, G, and B color components, one byte each), row by row (starting at the top), then column by column (starting at the left). For further details, see either the PPM file format specification, of the Wikipedia page about netpbm.

In order to create a PPM file whose SHA1 hash has the desired characteristics, one possibility is to change the least significant bits of the RGB components of the last pixels of the image.

5 What is provided

The following files contain examples and a reference implementation of parts of the assignment. Study/examine them all in detail!

`documentation/rfc3174.txt` — The description of the SHA1 secure hash algorithm.

`examples/deti_4_2_37_original.pdf` — An example of a PDF file.

`examples/deti_4_2_37_template.pdf` — An example of a PDF file with a comment near its end; compare it with the original file.

`examples/deti_4_2_37_final.pdf` — An example of a PDF file with an adjusted comment; compare it with the template file. Note that the SHA1 hash of this file

```
prompt> sha1sum examples/deti_4_2_37_final.pdf
f04350e83b1f1bb340ae2c81f7b9b10e00000000  examples/deti_4_2_37_final.pdf
```

ends with at least 8 hexadecimal digits that are all zero.

`examples/example_frame_001.ppm` — An example of a modified PPM file. Its SHA1 hash is

```
prompt> sha1sum examples/example_frame_001.ppm
d4b5136f18fc5d1873d4fbcc8d8d9acc00000000  examples/example_frame_001.ppm
```

It ends with at least 8 hexadecimal digits that are all zero.

`examples/example_frame_001_original.png` — The PPM file converted to a PNG file. Made with the following command:

```
prompt> convert -strip example_frame_001.ppm example_frame_001_original.png
```

(`convert` is a program of the ImageMagick image manipulation toolkit usually installed on GNU/Linux systems.)

`examples/example_frame_001_final.png` — The PNG file with a custom extra `tEXt` chunk. Its SHA1 hash is

```
prompt> sha1sum examples/example_frame_001_final.png
45f68fdc554bebbe4543626b790ddab100000000  examples/example_frame_001_final.png
```

It ends with at least 8 hexadecimal digits that are all zero.

`examples/example_movie.mkv` — A short movie. Its PPM individual frames, obtained using the command

```
prompt> ffmpeg -i example_movie.mkv '/tmp/frame_%03d.ppm'
```

have SHA1 hashes that end with at least 8 hexadecimal digits that are all zero.

parse_png.c — A simple program that can list the chunks of a PNG file. See this web page for more information about the structure of PNG files.

aad_data_types.h — Integer data types for the reference code. Included are also data the types needed for using Single Instruction Multiple Data (SIMD) instruction, for both Intel/AMD and Apple silicon processors.

aad_utilities.h — Code used by other parts of the program (time measurements, pseudo random number generation).

aad_png_crc.h — Each chunks of a PNG file ends with a 32-bit Cyclic Redundancy Checksum (CRC). This file contains code to compute it.

aad_ppm.h — This file contains code to read a PPM file to memory, padding it and the end with the SHA1 trailer (for convenience), and to write a PPM file.

aad_cuda_utilities.h — Code to initialize a CUDA device, load a kernel function, allocate memory, and run the kernel. It uses the CUDA driver API. It should be enough for the needs of this assignment.

aad_sha1.h — The implementation of the SHA1 secure hash update computation for a single 64-byte chunk. This is done as a customizable C preprocessor macro so that it can be adapted as needed. It can be used for the CPU code, with or without using SIMD instructions, and for the CUDA code. This is used in several places, so study it with extra care.

aad_sha1_cpu.h — Reference implementation of the SHA1 secure hash chunk update computation on the CPU (without and with SIMD instructions).

aad_sha1_cpu_tests.c — Program to test the correctness of the CPU code. Here the full SHA1 hash computation is performed; this includes initialization and padding at the end.

aad_sha1_cuda_kernel.cu — CUDA device code of the reference implementation of the SHA1 secure hash update computation.

aad_sha1_cuda_test.c — CPU code used to verify the correctness of the CUDA kernel code.

makefile — Makefile for the reference implementation. You will need to adjust the CUDA_ARCH line to reflect the CUDA device compute capabilities of your graphics card. Available targets:

```
clean           --- deletes not needed files (executables, ...)
sha1_cpu_tests  --- compiles the sha1_cpu_tests program
sha1_cuda_test  --- compiles the sha1_cuda_test program
parse_png       --- compiles the parse_png program
```

6 What is to be done

Create programs to modify PDF, PNG, and PPM files, and one extra file type of your choice. Each group should **try** doing as much as possible of the following:

mandatory Do it using a single CPU core without using SIMD instructions.

mandatory Do it using a single core using AVX or NEON instructions.

optional Do it using a single core using AVX2 and AVX512f (if available).

mandatory Do it using CUDA.

recommended Do it with SIMD instructions and OpenMP.

recommended Do it using a server and many clients.

recommended Do it with SIMD instructions and MPI.

recommended Do it using Web Assembly.

optional Do it using Web Assembly and SIMD instructions.

optional Do it using the SHA1 instructions available on Intel/AMD and on ARM processors.

optional How about an OpenCL implementation? oneAPI implementation? ROCm implementation? Metal implementation (for macOS)? Intel ISPC?

required For each of the above, measure how many attempts (SHA1 secure hash chunk update computations) you were able to do in one minute.

recommended Compare the performance, i.e., the numbers of attempts per minute, energy consumption, of as many computing devices as possible (recent and old computers, micro-controllers, smart phones, anything where you were able to run the program).

awesome Make a short movie whose PPM frames all have SHA1 hashes that end with 32 zero bits. The example film authored by the teacher was constructions using the following instructions (the 90 frames were originally named `frame_NNN.ppm_new` where NNN is the frame number)

```
2. mkdir x y
3. for n in {001..090}; do mv frame_${n}.ppm_new x/frame_${n}.ppm; done
4. ffmpeg -framerate 24 -pattern_type glob -i 'x/*.ppm' -vcodec ffv1 -level 3 -threads 1 \
          -coder 1 -context 1 -g 4 -slices 24 -slicecrc 1 example_movie.mkv
5. ffmpeg -i example_movie.mkv 'y/frame_%03d.ppm'
6. diff -rsq x y | grep -v identical
7. sha1sum y/*
```

optional/potentially awesome Something else (surprise the teacher).

7 Recommendations

The provided code computes the update of the SHA1 secure hash of a 64-byte message chunk. When one searches for a file pattern for which the SHA1 of the entire file has certain characteristics (such as 32 zero bits at its end), reprocessing should be done only on the chunks that have changed. Thus, it is advantageous to perform the file changes near its end, because in that case all but the last few file chunks need to change, and if so the partial SHA1 hash of chunks that do not change can be (pre)computed only once.

For CUDA kernels, because data transfer are relatively slow, try to transfer the minimum amount of information. In particular, the search results should use a common small memory area, and not a memory area for each thread. This can be done using atomic operations. So, create a memory area (on the host and on the device), and pass its device memory pointer to the CUDA kernel. Let us call it `storage_area`, and let us assume that it has 1024 `u32_t` words. The idea is that the first element of the buffer, `storage_area[0]`, holds the index of the first buffer word that is not yet in use. So, initialize it with 1 before each kernel run and copy that to the device. When a solution is found, increment `storage_area[0]` using an atomic instruction to reserve space for the solution data (1p integers in the following example, but that may change according to your needs) and, if the buffer is not yet full, copy the solution to the buffer:

```
idx = atomicAdd(storage_area,10u); // reserve space for one more solution (10 integer words)
if(idx < 1024u - 10u) // make sure we do not write outside of the buffer area
{
  storage_area[idx + 0] = solution[0];
  ...
  storage_area[idx + 9] = solution[9];
}
```

After a kernel run the device memory should be copied to the host memory and then all solutions found should be stored. Also, to avoid repeating work, make sure each CUDA tread is working on different parts of the search space.

8 Deliverable

One archive (either a compressed TAR archive or a ZIP file, **no RARs or other archive file formats**), containing

- the presentation (must be a PDF file), and
- all the source code (no executable files, please).
- the modified PDF, PNG, PPM files (no executable files, please).

The archive name must be of the form `NNNNNN_MMMMMM.extension` where `NNNNNN` and `MMMMMM` are student numbers (use as many as needed) and `extension` is the file extension (`tgz` or `zip`). Upload it for evaluation on `https://elearning.ua.pt/`.

The presentation must have a cover slide, with the names and respective student numbers, stating the percentage to time each one spent doing this assignment. All uses of AI should be clearly stated.

